# The system, for people who will change it.

One primary Bun application service and one SolidJS client. The server owns sessions, tools and permissions; feature-owned data stores remain separate. This is the package map, request path, extension surface, and the operational boundaries.

## The request path

```
browser → fetch /session/:id/message → HttpApi (packages/api) → SessionPrompt → ToolRegistry (one registry per session) → tool (read/grep/edit/bash/…) → Provider → model → events back over SSE → SessionProcessor persists parts → SQLite
```

### LAYERS, NOT A CONTAINER

Every service is an Effect layer. `LayerNode.make(layer, deps)` declares one, and `buildLayer(graph)` assembles the graph with a cycle check. 56 services live in the server package; the HTTP application graph is a single group that the routes, the task runner and the task generator all share — so an in-server worker has exactly the context a request handler has.

Providers are resolved per turn, not per process: a model id is looked up in the catalog, and **DiveeComposer** resolves to the concrete `provider/model` it routed to. The synthetic model carries the target's capabilities, so the UI's attachment buttons match the model that actually answers.

### DATA

The core application database stores sessions, messages, permissions, projects, and task state. Its v1/v2 message tables preserve the event-sourced conversation model; task execution adds a call journal and hook audit trail. Targeted Learning uses separate `books.db` and `student.db` stores for passages/vectors and student memory.

## The packages

| PACKAGE | HOLDS | NOTES |
|---|---|---|
| **server** | HTTP, sessions, tools, providers | Effect service graph and runtime orchestration |
| **db** | schema, migrations, providers, engine | Core SQLite persistence and adapters |
| **api** | HttpApi definitions | Typed schemas and OpenAPI contract |
| **app / ui** | SolidJS client and component library | Vite, shared themes and components |
| **learning** | Targeted Learning subsystem | Ingestion, OCR, hybrid retrieval, student memory and offline training |
| **llm** | Schema-first LLM client | Provider and protocol boundary |
| **plugin / plugins** | the plugin SDK and bundled plugins | hooks, tools, config |
| **memory / mcp / identity / sdk** | the rest | graph memory, MCP host, brand, clients |

# Running it, extending it, and the gaps.

## RUNNING IT

- **One process.** `bun run dev:both` starts the server (`:4097`) and the client. The server serves the built UI too.
- **Config precedence** is global → project → environment. The file is `diveeoi.json` (or `opencode.json`); both names are read, and the `diveeoi` name wins.
- **The database** defaults to the application data directory (channel-aware where configured); the legacy `OPENCODE_DB` override remains supported. Targeted Learning persists its library and student memory separately.
- **Authentication:** Basic and JWT/account modes are supported; the default is `DIVEEOI_AUTH_MODE=both`. Production requires explicit strong secrets, and registration is closed after the first account by default.
- **Verification:** run the current package test suites, TypeScript checks and production build for the exact source revision. Historical counts are in the worklog.

## BACKGROUND TASKS

A task is a session with a schedule. The runner polls two queues — tasks whose cron is due, and tasks left `queued` — runs them through the same prompt path an interactive session uses, and records the outcome. A call journal makes a crash recoverable: committed calls are replayed verbatim, a call that was in flight is surfaced as "interrupted, verify before retrying".

The agent drives them too, through `task_create`, `task_list`, `task_status`, `task_logs` and `task_control`. A task can ask the generator agent to write its own spec first — prompt, schedule, agent, models — and then runs unattended.

## EXTENSION POINTS

- **Agents** — `.opencode/agent/*.md` with frontmatter (model, mode, permission, tools).
- **Commands** — `.opencode/command/*.md`, with `$ARGUMENTS` and `$1`.
- **Skills** — `SKILL.md` folders, frontmatter with a name and a trigger description.
- **Plugins** — a `Plugin` export; hooks for config, events, tool execution, permissions.
- **MCP servers** — local or remote, declared in config; their tools join the registry.
- **Permissions** — per tool and per pattern, evaluated last-match-wins.

## THE GAPS, WITH THEIR COST

| GAP | WHAT IT MEANS |
|---|---|
| **No TUI** | the product is browser-first; there is no terminal client |
| **No built-in TLS** | terminate HTTPS at a reverse proxy such as nginx or Caddy |
| **Self-hosted operations** | you manage accounts/secrets, backups, upgrades, and external provider/MCP data paths |
| **No built-in TLS** | terminate at your own proxy; protect the DB file |
| **SQLite concurrency** | one writer at a time; fine for one operator |
| **Free-model ceiling** | the bundled tier is free and below a frontier model at hard reasoning |
| **Model churn** | the free tier moves; a model that fails is benched for a day |